

Smart Lender – Applicant Credibility Prediction for Loan Approval

Project Description:

Smart Lender is an intelligent loan approval prediction system that assists banks and financial institutions in evaluating loan applications using Machine Learning. The application predicts whether a loan should be approved based on the applicant's financial and personal details.

The system uses the Random Forest Machine Learning algorithm, which achieved the highest accuracy of **79%** during model evaluation. Users enter applicant details through a simple web interface developed using HTML and CSS. The Flask backend processes the input, scales the data using a trained Standard Scaler, and sends it to the trained Random Forest model for prediction.

The application improves the loan approval process by reducing manual effort, increasing prediction speed, minimizing human errors, and providing consistent decision support for loan officers.

Scenarios

Scenario 1: Loan Applicant

A customer visits the Smart Lender website and enters personal and financial details such as income, education, loan amount, loan term, credit history, and property area. The system predicts whether the loan is likely to be approved and immediately displays the result.

Scenario 2: Bank Loan Officer

A bank employee uses the application to evaluate multiple loan applications. Instead of manually checking each application, the system quickly predicts loan approval, helping officers make faster and more consistent decisions.

Scenario 3: Financial Institution

Financial institutions use Smart Lender as a decision-support system to reduce processing time, improve efficiency, and minimize risks while evaluating large numbers of loan applications.

Technical Architecture:

Description

Smart Lender follows a three-layer architecture consisting of the Presentation Layer, Application Layer, and Data Layer.

- The Presentation Layer provides a user-friendly interface developed using HTML and CSS where applicants enter their loan details.
- The Application Layer is implemented using the Flask framework. It validates user inputs, preprocesses the data, scales the features using the trained Standard Scaler, and invokes the Random Forest model for prediction.
- The Data Layer stores the trained machine learning model (model.pkl), scaler (scaler.pkl), and dataset used during model training.
- Docker is used to containerize the application for easy deployment across different environments.

This architecture ensures modularity, scalability, and efficient prediction performance.

Pre-requisites:

Before running the Smart Lender project, the following software and technologies should be installed:

- Python 3.10 or later
- Flask Framework
- NumPy
- Pandas
- Scikit-learn
- Joblib
- HTML and CSS knowledge
- Docker Desktop (for containerized deployment)
- Git and GitHub
- Visual Studio Code or PyCharm
- Web Browser (Chrome/Edge)

Project Workflow:

Milestone 1: Dataset Preparation and Model Selection

Activities

- Collect the loan prediction dataset.
- Analyze and clean missing values.
- Perform feature encoding and preprocessing.
- Split the dataset into training and testing datasets.
- Train multiple Machine Learning algorithms.

- Compare model performance.
- Select Random Forest as the final model based on highest accuracy.

Milestone 2: Machine Learning Model Development

Activities

- Train the Random Forest classifier.
- Evaluate using confusion matrix and classification report.
- Save the trained model as **model.pkl**.
- Save the Standard Scaler as **scaler.pkl** using Joblib.

Milestone 3: Flask Application Development

Activities

- Develop the Flask backend.
- Create routes for Home, Predict, and Submit pages.
- Load the trained model and scaler.
- Accept applicant information from HTML forms.
- Process predictions using the Random Forest model.
- Display loan approval results.

Milestone 4: Frontend Development

Activities

- Design responsive web pages using HTML and CSS.
- Create input forms for applicant details.
- Display prediction results clearly.
- Improve user experience with simple navigation.

Milestone 5: Deployment

Activities

- Create Docker configuration files.
- Containerize the Flask application.
- Install project dependencies.
- Deploy the application locally using Docker.
- Verify successful prediction through the web interface.

Milestone 6: Testing and Conclusion

Activities

- Test prediction accuracy.
- Validate user inputs.
- Verify application functionality.
- Check deployment success.
- Document results and future enhancements.

Milestone 1:

The first milestone focuses on preparing the loan prediction dataset and selecting the most suitable machine learning algorithm. Data preprocessing is one of the most important phases because the quality of the dataset directly affects the prediction accuracy. The loan prediction dataset contains applicant details such as gender, marital status, education, income, loan amount, credit history, and property area.

Initially, the dataset was analyzed to identify missing values, inconsistent records, and categorical attributes. Missing values were handled appropriately, while categorical features were converted into numerical values using encoding techniques. Feature scaling was then applied using the Standard Scaler to normalize the data before model training.

Several machine learning algorithms were trained and evaluated, including Logistic Regression, Decision Tree, K-Nearest Neighbors, Support Vector Machine, and Random Forest. After comparing the models based on prediction accuracy and reliability, Random Forest achieved the highest accuracy of approximately 79% and was selected as the final prediction model.

This milestone establishes the foundation of the Smart Lender application by ensuring clean, reliable, and high-quality data for machine learning.

loan_prediction.csv												
File View Tools Help												
Search in Drive												
Open with												
100%												
A	B	C	D	E	F	G	H	I	J	K	L	M
Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CosapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History	Property_Area	Loan_Status
LP001002	Male	No	0	Graduate	No	5849	0		360	1	Urban	Y
LP001003	Male	Yes	1	Graduate	No	4583	1508	128	360	1	Rural	N
LP001005	Male	Yes	0	Graduate	Yes	3000	0	66	360	1	Urban	Y
LP001006	Male	Yes	0	Not Graduate	No	2583	2358	120	360	1	Urban	Y
LP001008	Male	No	0	Graduate	No	6000	0	141	360	1	Urban	Y
LP001011	Male	Yes	2	Graduate	Yes	5417	4196	267	360	1	Urban	Y
LP001013	Male	Yes	0	Not Graduate	No	2333	1516	95	360	1	Urban	Y
LP001014	Male	Yes	3+	Graduate	No	3036	2504	158	360	0	Semiurban	N
LP001018	Male	Yes	2	Graduate	No	4006	1526	168	360	1	Urban	Y
LP001020	Male	Yes	1	Graduate	No	12841	10968	349	360	1	Semiurban	N
LP001024	Male	Yes	2	Graduate	No	3200	700	70	360	1	Urban	Y
LP001027	Male	Yes	2	Graduate		2500	1840	109	360	1	Urban	Y
LP001028	Male	Yes	2	Graduate	No	3073	8106	200	360	1	Urban	Y
LP001029	Male	No	0	Graduate	No	1853	2840	114	360	1	Rural	N
LP001030	Male	Yes	2	Graduate	No	1299	1086	17	120	1	Urban	Y
LP001032	Male	No	0	Graduate	No	4950	0	125	360	1	Urban	Y
LP001034	Male	No	1	Not Graduate	No	3596	0	100	240		Urban	Y
LP001036	Female	No	0	Graduate	No	3510	0	76	360	0	Urban	N
LP001038	Male	Yes	0	Not Graduate	No	4887	0	133	360	1	Rural	N
LP001041	Male	Yes	0	Graduate		2600	3500	115		1	Urban	Y
LP001043	Male	Yes	0	Not Graduate	No	7660	0	104	360	0	Urban	N
LP001046	Male	Yes	1	Graduate	No	5955	5625	315	360	1	Urban	Y
LP001047	Male	Yes	0	Not Graduate	No	2600	1911	116	360	0	Semiurban	N
LP001050		Yes	2	Not Graduate	No	3365	1917	112	360	0	Rural	N
LP001052	Male	Yes	1	Graduate		3717	2925	151	360		Semiurban	N
LP001066	Male	Yes	0	Graduate	Yes	9560	0	191	360	1	Semiurban	Y

Milestone 2:

Objective

After preprocessing the dataset, the next phase involved training and evaluating the machine learning model. The dataset was divided into training and testing sets to ensure unbiased model evaluation. Multiple classification algorithms were trained, and Random Forest was chosen due to its superior performance.

The trained Random Forest classifier learns complex relationships between applicant attributes and loan approval decisions. Model performance was evaluated using metrics such as accuracy score, confusion matrix, and classification report. Once the model achieved satisfactory results, it was saved using Joblib as model.pkl. Similarly, the trained Standard Scaler was saved as scaler.pkl to ensure that new user inputs are processed using the same scaling method as the training data.

Saving the trained model allows the Flask application to load it directly during runtime without retraining, improving the application's efficiency.

Random Forest Model

```
from sklearn.ensemble import RandomForestClassifier

# importing and building the random forest model
def RandomForest(X_train, X_test, y_train, y_test):
    model = RandomForestClassifier()
    model.fit(X_train, y_train)
    y_tr = model.predict(X_train)
    print(accuracy_score(y_tr, y_train))
    yPred = model.predict(X_test)
    print(accuracy_score(yPred, y_test))
    return model

# printing the train accuracy and test accuracy respectively
rf_model = RandomForest(X_train, X_test, y_train, y_test)
```

```
1.0
0.7885304659498208
```

```
best_model_name = results.loc[results['Test Accuracy'].idxmax(), 'Model']
print("Best performing model:", best_model_name)

print("\nClassification Report (Random Forest):")
print(classification_report(y_test, rf_model.predict(X_test)))

print("Confusion Matrix (Random Forest):")
print(confusion_matrix(y_test, rf_model.predict(X_test)))
```

Best performing model: Random Forest

Classification Report (Random Forest):

	precision	recall	f1-score	support
0	0.86	0.72	0.78	148
1	0.73	0.87	0.79	131
accuracy			0.79	279
macro avg	0.80	0.79	0.79	279
weighted avg	0.80	0.79	0.79	279

Confusion Matrix (Random Forest):

```
[[106  42]
 [ 17 114]]
```

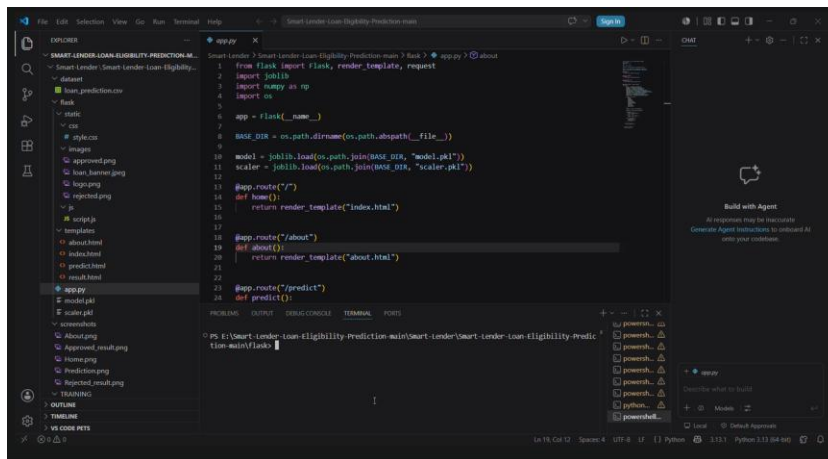
Milestone 3:

This milestone focuses on developing the backend of the Smart Lender application using the Flask framework. Flask acts as the bridge between the web interface and the machine learning model.

The application loads both the trained Random Forest model and Standard Scaler during startup. Flask defines different routes such as the Home page, Prediction page, and Result page. When a user submits loan details, Flask receives the input, converts it into numerical format, scales the data, and sends it to the Random Forest model for prediction.

The prediction result is then returned to the user through the web interface, displaying whether the loan is likely to be approved or rejected.

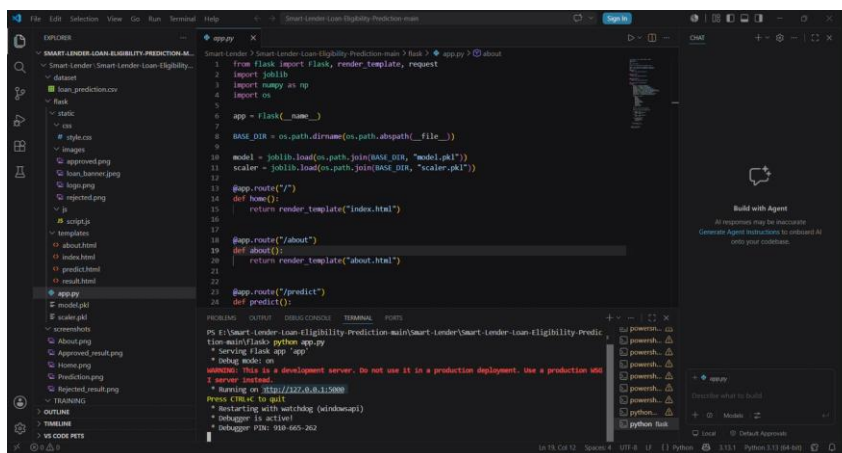
The backend was designed to be lightweight, efficient, and easy to maintain.



```

1 from flask import Flask, render_template, request
2 import joblib
3 import numpy as np
4 import os
5
6 app = Flask(__name__)
7
8 BASE_DIR = os.path.dirname(os.path.abspath(__file__))
9
10 model = joblib.load(os.path.join(BASE_DIR, "model.pkl"))
11 scaler = joblib.load(os.path.join(BASE_DIR, "scaler.pkl"))
12
13 @app.route("/")
14 def home():
15     return render_template("index.html")
16
17
18 @app.route("/about")
19 def about():
20     return render_template("about.html")
21
22
23 @app.route("/predict")
24 def predict():

```



```

PS E:\Smart-Lender-Loan-Eligibility-Prediction-main\Smart-Lender-Loan-Eligibility-Prediction-main> flask run
 * Serving Flask app "app"
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production web server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with watchdog (winapim)
 * Debugger is active!
 * Debugger PIN: 910-605-262

```

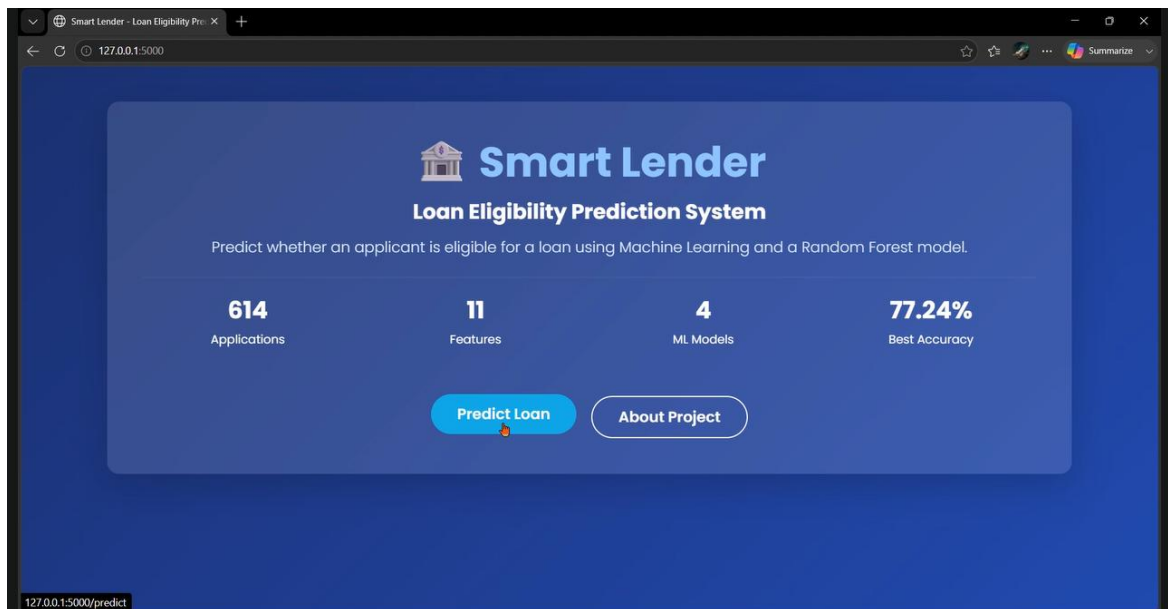
Milestone 4:

The frontend of Smart Lender was developed using HTML and CSS to provide an attractive and user-friendly interface. The main objective was to allow users to enter loan applicant information easily without requiring technical knowledge.

Separate HTML pages were created for the home page, prediction form, and result page. CSS styling was applied to improve readability and responsiveness. Form fields collect applicant information such as gender, income, education, loan amount, and credit history.

When the user clicks the Predict button, the entered information is securely transferred to the Flask backend for processing.

The frontend was designed with simplicity and usability in mind to improve the overall user experience.



Milestone 5:

After successful development and testing, the Smart Lender application was prepared for deployment. Docker was used to package the application together with all required dependencies, ensuring that the application behaves consistently across different environments.

The Docker container includes the Flask application, trained machine learning model, required libraries, and supporting files. Using Docker simplifies deployment by eliminating dependency conflicts and making the application portable.

The project source code was also managed using GitHub, enabling version control and collaborative development.

Deployment preparation ensures that the application can be easily executed on different systems with minimal configuration.

SmartLender / Dockerfile 

 manaswitha13 Add Dockerfile for Python Flask application

Code **Blame** 19 lines (13 loc) · 468 Bytes

```
1 FROM python:3.10-slim
2
3 WORKDIR /code
4
5 # Copy requirements and install dependencies
6 COPY "5.Project Development Phase/requirements.txt" /code/requirements.txt
7 RUN pip install --no-cache-dir -r requirements.txt
8
9 # Copy all project files
10 COPY . /code
11
12 # Grant all permissions so Hugging Face user can execute successfully
13 RUN chmod -R 777 /code
14
15 # Expose Hugging Face default port
16 EXPOSE 7860
17
18 # Run Flask application
19 CMD ["python", "5.Project Development Phase/app.py"]
```

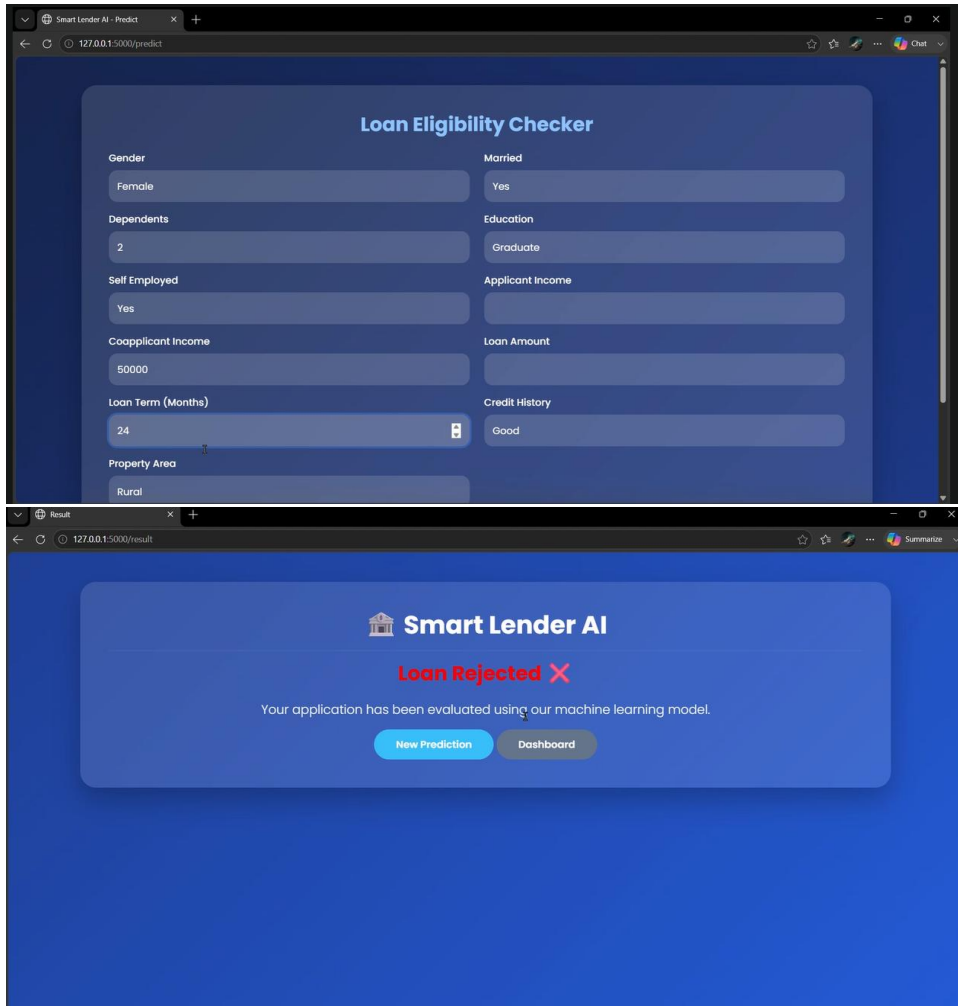
Milestone 6:

The final milestone focuses on validating the overall functionality and performance of the Smart Lender application. Functional testing was carried out using different applicant records to ensure that predictions are generated correctly.

Different combinations of applicant details were tested to verify the accuracy and reliability of the Random Forest model. Input validation was also performed to ensure that invalid data does not affect application performance.

The application successfully generated prediction results with low response time and stable performance. All major project objectives were achieved successfully, demonstrating the practical use of machine learning in loan approval prediction.

Future improvements include integrating a database, implementing user authentication, deploying the application to cloud platforms, and increasing prediction accuracy using larger datasets.



The image shows two screenshots of the Smart Lender AI web application. The top screenshot displays the 'Loan Eligibility Checker' form with the following inputs:

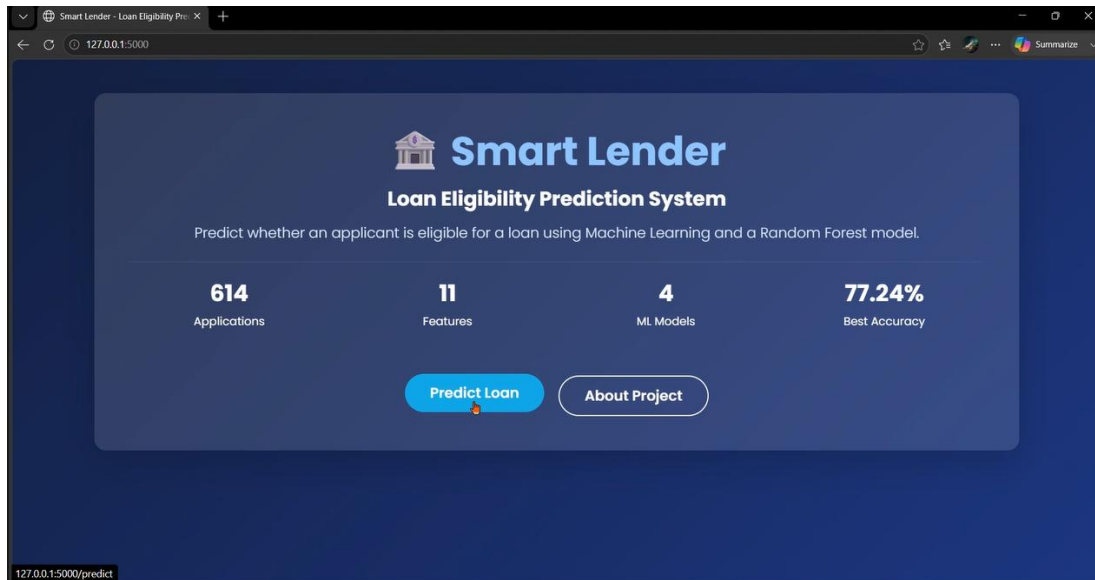
Field	Value
Gender	Female
Married	Yes
Dependents	2
Education	Graduate
Self Employed	Yes
Applicant Income	
Coapplicant Income	50000
Loan Amount	
Loan Term (Months)	24
Credit History	Good
Property Area	Rural

The bottom screenshot shows the 'Smart Lender AI' result page with a 'Loan Rejected' message and a 'New Prediction' button.

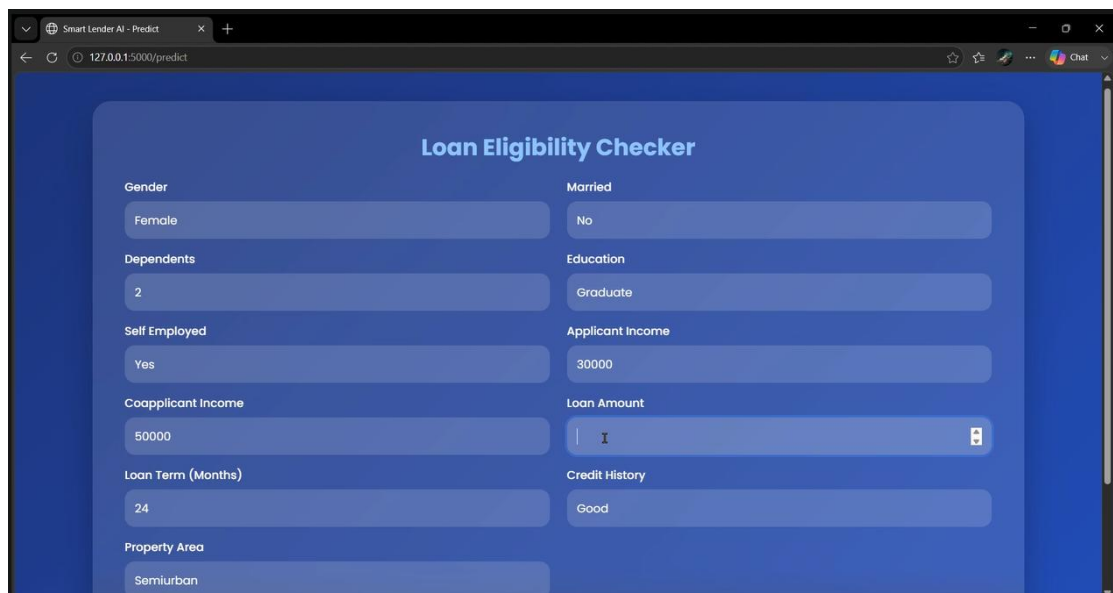
Exploring the Website's Web Pages:

Home / Generator Page:

Description: The single-page interface of CaptionAI serves as both the landing page and the generation tool. It features a dark glassmorphism design with animated background blobs, a header displaying the CaptionAI brand (wand-magic-sparkles icon + logo), and a tagline. The main section contains the input form, platform toggle, tone selector, and the AI-generated results panel all on one seamless page.

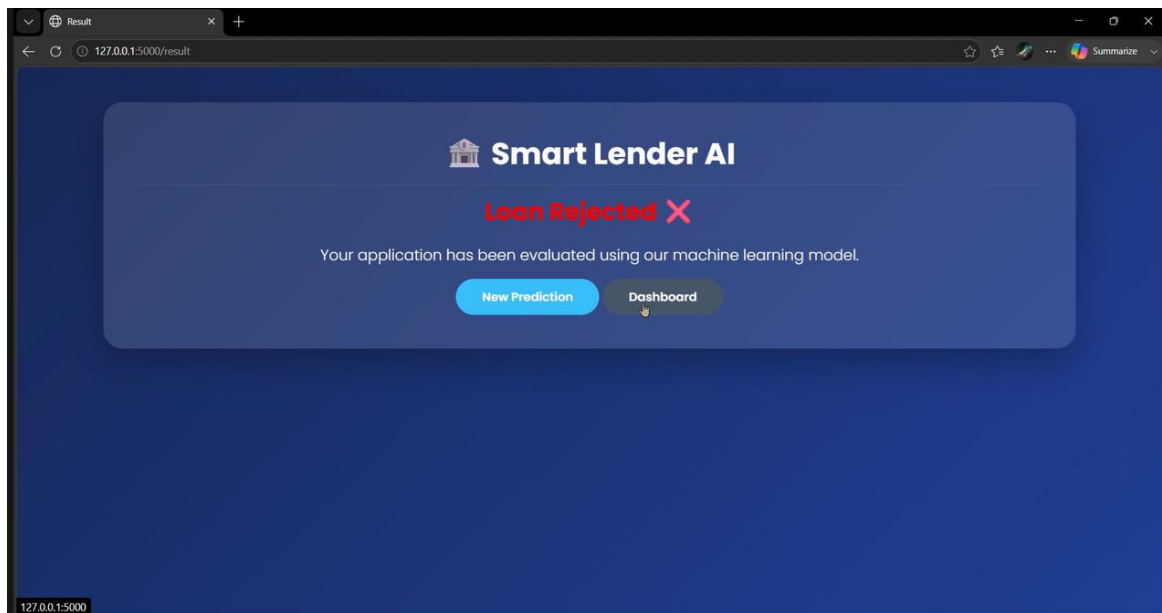


Input Section:



The screenshot shows the 'Loan Eligibility Checker' form. It has a dark blue background and a white border. The form is divided into two columns. The left column contains: 'Gender' (Female), 'Dependents' (2), 'Self Employed' (Yes), 'Coapplicant Income' (50000), 'Loan Term (Months)' (24), and 'Property Area' (Semiurban). The right column contains: 'Married' (No), 'Education' (Graduate), 'Applicant Income' (30000), 'Loan Amount' (input field with a currency symbol), and 'Credit History' (Good).

Results Section:



Conclusion:

Conclusion

Smart Lender successfully demonstrates the application of Machine Learning in the banking and financial sector by providing fast and reliable loan approval predictions. Using the Random Forest algorithm with an accuracy of **79%**, the system assists banks in reducing manual verification effort and improving decision-making efficiency.

The Flask-based web application offers a simple and interactive interface, while Docker enables easy deployment across different environments. The project showcases the practical integration of Machine Learning, Flask, and web technologies into a real-world financial application. Future enhancements may include user authentication, database integration, cloud deployment, and advanced credit risk analysis to further improve prediction accuracy and usability.